

Zagajenia wstępne do laboratorium Podstawy informatyki i programowania

LABORATORIUM 1: C++ – Podstawy algorytmiki i tablic

1. **Indeksowanie struktur danych:** Określ dopuszczalny zakres indeksowania tablicy względem jej rozmiaru N . Jakie są skutki odwołania się do elementu spoza tego zakresu?
 2. **Definiowanie rozmiaru:** Wyjaśnij, dlaczego w definicji tablic statycznych rozmiar musi być stałą znaną na etapie kompilacji, a nie zmienną pobieraną w trakcie pracy programu.
 3. **Specyfika dzielenia całkowitoliczbowego:** Opisz wynik operacji dzielenia liczby nieparzystej przez liczbę 2 w przypadku, gdy obie są typu `int`. Jak wymusić poprawny wynik zmiennoprzecinkowy?
 4. **Logika inicjalizacji ekstremów:** Wyjaśnij błąd logiczny polegający na inicjowaniu zmiennej przechowującej wartość ekstremalną liczbą spoza dziedziny danych (np. inicjowanie maksimum zerem dla liczb ujemnych).
 5. **Formatowanie wyjścia:** Jakie mechanizmy biblioteczne pozwalają na kontrolę precyzji wyświetlania liczb rzeczywistych w strumieniu wyjściowym?
 6. **Iteracja:** Porównaj zastosowanie pętli o znanej liczbie powtórzeń z pętlą warunkową w kontekście przeglądania całego zbioru danych.
 7. **Przekazywanie argumentów:** Wyjaśnij, dlaczego funkcja operująca na tablicy klasycznej musi otrzymywać informację o jej rozmiarze jako dodatkowy parametr.
 8. **Determinizm losowania:** Dlaczego bez zainicjowania ziarna generatora liczb pseudolosowych program przy każdym uruchomieniu wygeneruje te same wartości?
 9. **Warunek wyznaczania mediany:** Opisz niezbędny krok w przetwarzaniu zbioru danych, bez którego wyznaczenie wartości środkowej (mediany) jest niemożliwe.
 10. **Precyzja typów:** Omów różnice między typami zmiennoprzecinkowymi o pojedynczej i podwójnej precyzji w kontekście dokładności obliczeń inżynierskich.
-

LABORATORIUM 2: Python – Automatyzacja i operacje na plikach

1. **Cykl życia kodu:** Wyjaśnij różnicę między procesem kompilacji a interpretacji kodu źródłowego.
2. **Bezpieczeństwo zasobów (I/O):** Dlaczego w operacjach na plikach zaleca się stosowanie menedżerów kontekstu (`with`), a nie ręczne zamykanie uchwytów do plików?

3. **Charakterystyka typowania:** Omów koncepcję dynamicznego typowania zmiennych w języku Python i jej wpływ na sposób deklaracji zmiennych.
 4. **Konwersja typów danych:** Opisz procedurę przekształcania danych tekstowych odczytanych z pliku na wartości liczbowe umożliwiające wykonywanie operacji matematycznych.
 5. **Parsowanie tekstu:** Wyjaśnij rolę metod służących do usuwania białych znaków oraz dzielenia ciągów tekstowych względem zadanego separatora podczas wczytywania plików CSV.
 6. **Elastyczność struktur danych:** Porównaj listy w języku Python z tablicami statycznymi pod kątem możliwości zmiany ich objętości w trakcie działania programu.
 7. **Mechanizmy odporności na błędy:** Opisz działanie bloków służących do przechwytywania wyjątków w sytuacjach awaryjnych (np. próba otwarcia nieistniejącego zasobu).
 8. **Formatowanie łańcuchów znaków:** Omów nowoczesne metody wstawiania wartości zmiennych do ciągów tekstowych z jednoczesnym określeniem ich precyzji wyświetlania.
 9. **Struktura danych CSV:** Zdefiniuj format CSV i wyjaśnij znaczenie separatora w kontekście poprawnej interpretacji kolumn danych.
 10. **Obsługa czasu systemowego:** Jakie biblioteki standardowe pozwalają na rejestrację momentu wykonania obliczeń lub generowania raportu?
-

LABORATORIUM 3: Python – Analiza numeryczna i wizualizacja

1. **Wydajność obliczeń seryjnych:** Wyjaśnij koncepcję wektoryzacji i jej wpływ na czas wykonywania operacji matematycznych na dużych zbiorach danych w porównaniu do pętli iteracyjnych.
2. **Generowanie dziedziny funkcji:** Porównaj metody tworzenia ciągów liczbowych o zadanej liczbie elementów z metodami o zadanym kroku przyrostu.
3. **Standard wykresu technicznego:** Wymień niezbędne składowe opisu graficznego, które pozwalają na jednoznaczną interpretację wielkości fizycznych przedstawionych na wykresie.
4. **Identyfikacja serii danych:** Opisz rolę legendy na wykresie w przypadku prezentacji więcej niż jednego przebiegu w tym samym układzie współrzędnych.
5. **Eksport danych graficznych:** Jakie czynniki decydują o wyborze formatu pliku (np. PNG vs PDF) przy zapisywaniu wygenerowanych wizualizacji?
6. **Złożone układy współrzędnych:** Wyjaśnij cel stosowania podwykresów (ang. *subplots*) w ramach jednego okna graficznego.
7. **Semantyka operacji tablicowych:** Opisz skutek wykonania operacji mnożenia tablicy numerycznej przez skalar w porównaniu do analogicznej operacji na standardowej liście.
8. **Dobór formy wizualizacji:** Kiedy w praktyce inżynierskiej stosuje się wykres punktowy, a kiedy liniowy do prezentacji wyników pomiarowych?

9. **Analiza siatkowa:** Wyjaśnij funkcję siatki pomocniczej na wykresie w kontekście odczytywania wartości pośrednich sygnału.
 10. **Reprezentacja dyskretna sygnału:** Dlaczego gęstość próbkowania na osi czasu ma krytyczne znaczenie dla poprawności wizualizacji sygnałów okresowych?
-

LABORATORIUM 4: Projektowanie i dokumentacja techniczna

1. **Modelowanie procesów:** Opisz znaczenie poszczególnych symboli graficznych stosowanych w schematach blokowych algorytmów (operacja, warunek, wejście/wyjście).
 2. **Strukturyzacja projektu:** Wyjaśnij, w jaki sposób podział programu na niezależne moduły funkcjonalne ułatwia jego testowanie i rozwój.
 3. **Walidacja wejścia:** Dlaczego program inżynierski powinien weryfikować dane przed przystąpieniem do obliczeń (np. sprawdzać dziedzinę funkcji)?
 4. **Architektura dokumentacji:** Wymień kluczowe rozdziały, które powinny znaleźć się w profesjonalnym raporcie technicznym dotyczącym oprogramowania.
 5. **Dokumentacja wewnątrz kodu:** Opisz rolę komentarzy dokumentujących (np. Docstrings) w kontekście czytelności kodu dla innych programistów.
 6. **Koncepcja MVP:** Wyjaśnij znaczenie budowania w pierwszej kolejności minimalnej działającej wersji produktu przed dodawaniem zaawansowanych funkcji estetycznych.
 7. **Logika przepływu danych:** Opisz na schemacie blokowym ścieżkę komunikacji między modułem wczytywania danych a modułem obliczeniowym.
 8. **Uzasadnienie teoretyczne:** Dlaczego opis matematyczny/fizyczny problemu musi poprzedzać prezentację kodu źródłowego w dokumentacji projektu?
 9. **Weryfikacja poprawności:** Na czym polega opracowanie zestawu danych testowych i jakie sytuacje (np. brzegowe) powinny one obejmować?
 10. **Standardy jakości kodu:** Wymień cechy, które decydują o tym, że kod źródłowy uznaje się za profesjonalny i zgodny z dobrą praktyką inżynierską.
-

LABORATORIUM 5: Obrona i weryfikacja projektów

1. **Analiza zgodności (V&V):** Wyjaśnij różnicę między weryfikacją (czy program działa zgodnie z kodem) a walidacją (czy program poprawnie rozwiązuje problem fizyczny).
2. **Transparentność inżynierska:** Dlaczego dokumentacja techniczna musi być sporządzona w sposób umożliwiający powtórzenie obliczeń przez osobę trzecią?
3. **Integralność dokumentów:** Omów zalety stosowania nieedytowalnych formatów plików (np. PDF) przy przekazywaniu finalnej dokumentacji technicznej.

4. **Odporność na błędy operatora:** Jak powinna zareagować stabilna aplikacja w przypadku wprowadzenia przez użytkownika danych o błędnym typie?
5. **Interpretacja anomalii:** O czym mogą świadczyć nietypowe wartości ekstremalne lub nagłe zmiany na wykresach danych rzeczywistych?
6. **Optymalizacja końcowa:** Na czym polega proces refaktoryzacji kodu przed oddaniem projektu do użytku?
7. **Znaczenie jednostek miar:** Wyjaśnij skutki braku określenia jednostek fizycznych w raporcie technicznego lub na osiach wykresów.
8. **Przenaszalność projektu:** Opisz zasady przygotowania paczki z projektem, aby uniknąć problemów z błędnymi ścieżkami dostępu do plików na innym komputerze.
9. **Efektywność przekazu:** Porównaj czytelność prezentacji dużej ilości danych surowych w formie tabelarycznej oraz graficznej.
10. **Etyka i odpowiedzialność:** Dlaczego poprawność logiczna algorytmu obliczeniowego jest ważniejsza niż walory wizualne interfejsu w oprogramowaniu inżynierskim?

KARTA ĆWICZENIA LABORATORYJNEGO NR 1

Przedmiot: Podstawy informatyki i programowania

Kierunek: Elektrotechnika

Temat: Implementacja algorytmów przetwarzania danych pomiarowych w języku C++.

Zarządzanie pamięcią i struktury danych.

Wymiar godzin: 4 godz. lekcyjne

Rodzaj zajęć: Laboratorium programistyczne

1. CEL ĆWICZENIA

Celem zajęć jest praktyczna weryfikacja umiejętności programowania strukturalnego w języku C++ oraz nabycie kompetencji w zakresie niskopoziomowej obróbki danych inżynierskich. Ćwiczenie kładzie nacisk na poprawność obliczeń numerycznych oraz optymalne zarządzanie zasobami pamięci operacyjnej.

Cele szczegółowe obejmują:

- **Opanowanie metod akwizycji danych:** Student uczy się implementacji procedur wprowadzania danych oraz ich przechowywania w tablicowych strukturach danych.
- **Modularyzację kodu źródłowego:** Nabycie umiejętności dekompozycji złożonego problemu na mniejsze jednostki funkcjonalne (funkcje) w celu podniesienia czytelności i reużywalności kodu.
- **Zarządzanie precyzją obliczeń:** Zrozumienie specyfiki arytmetyki komputerowej, ze szczególnym uwzględnieniem operacji na typach zmiennoprzecinkowych (`double`) oraz unikania błędów dzielenia całkowitoliczbowego.
- **Implementację algorytmów bezbibliotecznych:** Samodzielne opracowanie procedur wyszukiwania ekstremów i sortowania w celu pełnego zrozumienia mechanizmów kontroli przepływu danych.

2. WPROWADZENIE MERYTORYCZNE

W inżynierii dane pomiarowe reprezentowane są jako dyskretne próbki przechowywane w strukturach tablicowych. Przetwarzanie takich zbiorów wymaga znajomości algorytmów iteracyjnych oraz dbałości o bezpieczeństwo dostępu do pamięci.

Kluczowe aspekty techniczne:

- **Indeksowanie tablic:** Zrozumienie statycznej alokacji pamięci i dostęp do elementów w ścisłym zakresie od `0` do `N-1`.

- **Arytmetyka komputerowa:** Dbłość o precyzję obliczeń inżynierskich poprzez unikanie błędów zaokrągleń oraz stosowanie rzutowania typów (*casting*) w operacjach dzielenia.
- **Algorytmika niskopoziomowa:** Samodzielna implementacja procedur analitycznych, co jest kluczowe w programowaniu systemów o ograniczonych zasobach (np. systemy wbudowane).
- **Strukturalizacja programu:** Wykorzystanie funkcji do odseparowania logiki akwizycji danych od logiki obliczeniowej i prezentacji wyników.

3. TREŚĆ ZADANIA

Zadanie polega na zaprojektowaniu i zaimplementowaniu aplikacji konsolowej, która symuluje działanie modułu analizy danych pomiarowych. Program powinien umożliwiać zebranie serii N pomiarów (np. wartości napięcia), wykonanie ich analizy statystycznej oraz przejrzyste wyświetlenie wyników.

Etapy realizacji projektu:

1. **Akwizycja danych (Input):** Implementacja mechanizmu wprowadzania danych do tablicy statycznej (manualnie lub za pomocą generatora).
 2. **Przetwarzanie algorytmiczne (Logic):** Opracowanie funkcji wyznaczających parametry statystyczne zbioru oraz procedur porządkujących dane.
 3. **Prezentacja wyników (Output):** Wyświetlenie sformatowanych danych oraz wyników analizy w sposób czytelny, z zachowaniem standardów inżynierskich (precyzja miejsc po przecinku).
-

4. WYMAGANIA I KRYTERIA OCENY

Ocena 3.0 (Dostateczny)

- Poprawna deklaracja tablicy o stałym rozmiarze (np. dla 10 elementów).
- Implementacja pętli umożliwiającej użytkownikowi ręczne wprowadzenie danych z klawiatury.
- Wyświetlenie wszystkich wprowadzonych danych w czytelnej formie.
- Brak błędów kompilacji.

Ocena 3.5 (Dostateczny Plus)

- Spełnienie wymagań na ocenę 3.0.
- Implementacja algorytmu obliczającego średnią arytmetyczną próbek.
- Zastosowanie rzutowania typów (*casting*) w operacji dzielenia.

- Formatowanie wyniku do 2 miejsc po przecinku (biblioteka `iomanip`).

Ocena 4.0 (Dobry)

- Spełnienie wymagań na ocenę 3.5.
- Implementacja algorytmu wyszukiwania wartości minimalnej i maksymalnej w zbiorze.
- Poprawna inicjalizacja zmiennych pomocniczych (pierwszym elementem tablicy, a nie zerem).
- Zastosowanie znaczących nazw zmiennych i podstawowych komentarzy w kodzie.

Ocena 4.5 (Dobry Plus)

- Spełnienie wymagań na ocenę 4.0.
- Implementacja własnego algorytmu sortowania (np. bąbelkowe lub przez wybieranie). Zakaz użycia `std::sort`.
- Wyznaczenie mediany (wartości środkowej) z uwzględnieniem parzystej i nieparzystej liczby elementów.
- Prezentacja zbioru przed i po sortowaniu.

Ocena 5.0 (Bardzo Dobry)

- Spełnienie wymagań na ocenę 4.5.
- Pełna dekompozycja programu na funkcje (np. funkcje do wczytywania, obliczeń i wyświetlania).
- Implementacja generatora liczb pseudolosowych do automatycznego wypełniania tablicy danymi z zadanego zakresu (np. 0.0 - 5.0).
- Obsługa podstawowych błędów wejścia (np. blokada przed wpisaniem znaku zamiast liczby).

5. WSKAZÓWKI DLA STUDENTA

- **Losowanie danych:** Użyj funkcji `rand()` skalując jej wynik: $\text{Min} + (\text{double})\text{rand()} / \text{RAND_MAX} * (\text{Max} - \text{Min})$. Pamiętaj o jednokrotnym wywołaniu `srand(time(NULL))`.
- **Przekazywanie do funkcji:** Pamiętaj, że funkcja operująca na tablicy musi przyjmować tablicę oraz jej rozmiar jako osobny argument.
- **Mediana:** Wartość ta wymaga uprzedniego posortowania zbioru. Dla parzystej liczby elementów jest to średnia z dwóch środkowych próbek o indeksach $(N/2)-1$ oraz $N/2$.
- **Formatowanie:** Wykorzystaj `std::fixed` i `std::setprecision(2)` do estetycznej prezentacji wyników inżynierskich.

KARTA ĆWICZENIA LABORATORYJNEGO NR 2

Przedmiot: Podstawy informatyki i programowania

Kierunek: Elektrotechnika

Temat: Migracja algorytmów inżynierskich do środowiska Python. Przetwarzanie danych seryjnych i operacje na plikach (I/O).

Wymiar godzin: 4 godz. lekcyjne

Rodzaj zajęć: Laboratorium programistyczne

1. CEL ĆWICZENIA

Głównym celem zajęć jest nabycie kompetencji w zakresie wykorzystania języka skryptowego Python jako narzędzia do szybkiego prototypowania algorytmów oraz automatyzacji procesów obróbki dużych zbiorów danych inżynierskich. Ćwiczenie ma na celu pokazanie korzyści płynących z wyższego poziomu abstrakcji przy jednoczesnym zachowaniu rygoru poprawnej logiki obliczeniowej.

Cele szczegółowe obejmują:

- **Analizę porównawczą paradygmatów:** Student uczy się rozpoznawać różnice między językiem kompilowanym (C++) a interpretowanym (Python), ze szczególnym uwzględnieniem dynamicznego typowania.
- **Implementację mechanizmów trwałości danych:** Opanowanie metod otwierania, odczytu i zapisu plików tekstowych oraz formatów CSV, co jest kluczowe w pracy z logami urządzeń pomiarowych.
- **Opracowanie procedur automatyzacji:** Nabycie umiejętności tworzenia skryptów zdolnych do masowego przetwarzania plików bez ingerencji operatora (*batch processing*).
- **Zarządzanie błędami i wyjątkami:** Nauka obsługi sytuacji nieprzewidzianych (np. brak pliku, błędny format danych) przy użyciu mechanizmów `try-except`, co podnosi niezawodność oprogramowania.

2. WPROWADZENIE MERYTORYCZNE

W procesie badawczym inżynier często spotyka się z sytuacją, w której dane pomiarowe są generowane w formie plików tekstowych o znacznej objętości. Język Python, dzięki swojej składni oraz wbudowanym mechanizmom zarządzania kolekcjami, pozwala na znaczące skrócenie czasu implementacji systemów analizujących takie zbiory.

Kluczowe aspekty techniczne:

- **Struktury danych (Lists/Dictionaries):** W przeciwieństwie do tablic C++, listy w Pythonie są dynamiczne, co pozwala na gromadzenie danych o nieznanym z góry rozmiarze.
- **Strumieniowe operacje na plikach:** Wykorzystanie konstrukcji `with open()`, która gwarantuje poprawne zamknięcie uchwytu do pliku i bezpieczeństwo zasobów systemowych.
- **Parsowanie danych (Data Parsing):** Proces konwersji surowego ciągu znaków (*string*) na wartości numeryczne (`float` / `int`) wraz z usuwaniem nagłówek i zbędnych separatorów.
- **Interpretacja vs Kompilacja:** Zrozumienie natury wykonywania kodu instrukcja po instrukcji, co ułatwia debugowanie i testowanie algorytmów w czasie rzeczywistym.

3. TREŚĆ ZADANIA

Zadanie polega na stworzeniu skryptu analizującego zewnętrzny zbiór danych (np. logi napięć wyeksportowane z symulatora lub oscyloskopu). Student musi zaimplementować „potok przetwarzania” (*pipeline*), który przekształci surowy plik wejściowy w czytelny raport końcowy.

Etapy realizacji projektu:

1. **Akwizycja z plików (Input):** Lokalizacja i otwarcie pliku tekstowego zawierającego dane liczbowe, z uwzględnieniem pominięcia linii nagłówkowych.
2. **Przetwarzanie logiczne (Logic):** Wykonanie operacji statystycznych z wykorzystaniem mechanizmów Pythona (np. funkcje wbudowane w obiekty list) oraz filtracja rekordów uszkodzonych.
3. **Generowanie raportu (Output):** Automatyczny zapis wyników analizy do nowego pliku technicznego wraz z podsumowaniem statystycznym i znacznikiem czasu.

4. WYMAGANIA I KRYTERIA OCENY

Ocena 3.0 (Dostateczny)

- Poprawna instalacja i konfiguracja środowiska (np. VS Code, PyCharm lub IDLE).
- Implementacja skryptu, który pobiera od użytkownika serię liczb i przechowuje je w liście.
- Wykorzystanie pętli `for` do wyświetlenia zawartości listy oraz funkcji `len()` i `sum()`.
- Brak błędów składniowych (*Syntax Errors*).

Ocena 3.5 (Dostateczny Plus)

- Spełnienie wymagań na ocenę 3.0.
- Implementacja samodzielnej funkcji obliczającej średnią z listy pomiarów.

- Zastosowanie formatowania wyjścia (*f-strings*) do prezentacji wyników z określoną precyzją (np. do 3 miejsc po przecinku).
- Podział logiki programu na sekcję wprowadzania danych i sekcję obliczeniową.

Ocena 4.0 (Dobry)

- Spełnienie wymagań na ocenę 3.5.
- **Praca z plikiem:** Skrypt poprawnie otwiera plik tekstowy (np. `dane.txt`), wczytuje z niego liczby i zapisuje do listy.
- Implementacja obsługi wyjątków (`try-except`) na wypadek braku pliku lub błędnych danych.
- Wyszukiwanie wartości ekstremalnych (min/max) z wykorzystaniem metod wbudowanych języka.

Ocena 4.5 (Dobry Plus)

- Spełnienie wymagań na ocenę 4.0.
- **Parsowanie CSV:** Obsługa plików z wieloma kolumnami danych oddzielonymi przecinkiem lub średnikiem.
- Generowanie pliku wynikowego (`raport.txt`), zawierającego statystyki oraz datę wykonania (biblioteka `datetime`).
- Implementacja prostego sortowania listy danych (użycie metody `sort()` lub funkcji `sorted()`).

Ocena 5.0 (Bardzo Dobry)

- Spełnienie wymagań na ocenę 4.5.
- **Modularyzacja:** Kod podzielony na funkcje z jasno zdefiniowanymi parametrami i wartościami zwrótnymi.
- Implementacja mechanizmu czyszczenia danych (*Data Cleaning*) – ignorowanie pustych linii lub błędnych wpisów.
- Automatyczne generowanie nazwy raportu na podstawie nazwy pliku wejściowego.
- Deklaracja tematu projektu końcowego i wstępny zarys algorytmu.

5. WSKAZÓWKI DLA STUDENTA

- **Zarządzanie plikami:** Zawsze używaj konstrukcji `with open('nazwa.txt', 'r') as f:` , aby uniknąć blokowania zasobów w systemie operacyjnym.

- **Konwersja typów:** Pamiętaj, że dane odczytane z pliku są domyślnie typu `string`. Użyj `float(zmienna)` przed wykonaniem operacji matematycznych.
- **Metody tekstowe:** Funkcja `strip()` usuwa znaki nowej linii (`\n`), a `split(',')` dzieli linię na poszczególne wartości (kolumny).
- **Pomoc systemowa:** Korzystaj z polecenia `help(lista.append)` lub `print(dir(lista))` w konsoli Pythona, aby sprawdzić dostępne metody struktur danych.

KARTA ĆWICZENIA LABORATORYJNEGO NR 3

Przedmiot: Podstawy informatyki i programowania

Kierunek: Elektrotechnika

Temat: Wizualizacja danych pomiarowych i obliczenia numeryczne w środowisku Python.

Wykorzystanie bibliotek NumPy i Matplotlib.

Wymiar godzin: 4 godz. lekcyjne

Rodzaj zajęć: Laboratorium programistyczne

1. CEL ĆWICZENIA

Celem zajęć jest nabycie umiejętności w zakresie graficznej prezentacji wyników badań inżynierskich oraz wykorzystania specjalistycznych bibliotek numerycznych do analizy danych wektorowych. Ćwiczenie kładzie nacisk na standardy tworzenia wykresów technicznych oraz optymalizację obliczeń seryjnych.

Cele szczegółowe obejmują:

- **Opanowanie metod wektoryzacji obliczeń:** Student uczy się zastępowania klasycznych pętli operacjami macierzowymi biblioteki NumPy, co znacząco podnosi wydajność przetwarzania sygnałów.
- **Implementację standardów wizualizacji inżynierskiej:** Nabycie umiejętności tworzenia profesjonalnych wykresów zawierających opisy osi, jednostki, siatki pomiarowe oraz legendy zgodnie z zasadami dokumentacji technicznej.
- **Integrację potoku przetwarzania danych:** Połączenie umiejętności odczytu danych z plików (z Lab 2) z ich graficzną interpretacją i analizą numeryczną.
- **Symulację procesów fizycznych:** Wykorzystanie narzędzi informatycznych do modelowania zjawisk (np. przebiegów prądowych i napięciowych) oraz ich weryfikacji graficznej.

2. WPROWADZENIE MERYTORYCZNE

W pracy inżyniera sama wartość liczbowa jest często niewystarczająca do poprawnej interpretacji zjawiska. Kluczowe jest przedstawienie danych w formie wykresów czasowych, charakterystyk częstotliwościowych lub statystycznych (histogramy).

Kluczowe aspekty techniczne:

- **NumPy (Numerical Python):** Biblioteka dostarczająca strukturę `ndarray`, która pozwala na wykonywanie operacji matematycznych na całych zbiorach danych jednocześnie (obliczenia wektorowe).

- **Matplotlib (Pyplot):** Standardowe narzędzie do tworzenia wizualizacji. Pozwala na pełną kontrolę nad estetyką wykresu, co jest niezbędne przy generowaniu grafik do raportów i publikacji technicznych.
- **Standardy wykresu technicznego:** Wykres inżynierski musi być jednoznaczny. Wymagane jest stosowanie skali liniowych lub logarytmicznych, opisu wielkości fizycznych wraz z ich jednostkami oraz dbałość o czytelność (odpowiedni dobór kolorów i grubości linii).
- **Generowanie przebiegów syntetycznych:** Umiejętność tworzenia wektorów czasu i wartości funkcji (np. sinus, przebieg prostokątny) do celów porównawczych z danymi rzeczywistymi.

3. TREŚĆ ZADANIA

Zadanie polega na opracowaniu skryptu, który dokona wizualizacji i analizy danych pomiarowych. Student musi zaimplementować funkcjonalności pozwalające na porównanie danych teoretycznych (z modelu matematycznego) z danymi rzeczywistymi (wczytanymi z pliku).

Etapy realizacji projektu:

1. **Przygotowanie danych (Input):** Generowanie wektorów teoretycznych przy użyciu funkcji NumPy (np. `linspace`, `sin`, `exp`) oraz wczytanie danych zaszumionych z pliku opracowanego w poprzednim laboratorium.
2. **Analiza numeryczna (Logic):** Obliczenie parametrów charakterystycznych dla przebiegów (np. wartość skuteczna, całka numeryczna metodą trapezów lub wyznaczenie punktów przecięcia z zerem).
3. **Wizualizacja zaawansowana (Output):** Stworzenie zestawienia wykresów (np. w układzie `subplots`), naniesienie wyników obliczeń bezpośrednio na wykres oraz eksport grafiki do formatu bezstratnego (PDF/PNG).

4. WYMAGANIA I KRYTERIA OCENY

Ocena 3.0 (Dostateczny)

- Poprawna instalacja i import bibliotek `numpy` oraz `matplotlib.pyplot`.
- Wygenerowanie prostego wykresu funkcji matematycznej (np. $y = x^2$) dla zadanego zakresu.
- Wyświetlenie wykresu w oknie interaktywnym.
- Brak błędów logicznych w definicji dziedziny funkcji.

Ocena 3.5 (Dostateczny Plus)

- Spełnienie wymagań na ocenę 3.0.

- Dodanie podstawowych elementów opisu technicznego: etykiety osi X i Y, tytuł wykresu oraz siatka (`grid`).
- Zastosowanie biblioteki NumPy do wygenerowania wektora argumentów (użycie `np.linspace` lub `np.arange`).
- Zdefiniowanie parametrów fizycznych wykresu (np. jednostki w nawiasach kwadratowych).

Ocena 4.0 (Dobry)

- Spełnienie wymagań na ocenę 3.5.
- **Wizualizacja danych zewnętrznych:** Skrypt wczytuje dane z pliku (np. `dane.csv`) i nanosi je na wykres w formie punktowej (`scatter` lub odpowiedni marker).
- Naniesienie na ten sam wykres linii trendu lub modelu teoretycznego w celu porównania.
- Dodanie legendy (`legend`) identyfikującej poszczególne serie danych.

Ocena 4.5 (Dobry Plus)

- Spełnienie wymagań na ocenę 4.0.
- **Układy wielowykresowe:** Zastosowanie funkcji `plt.subplots()` do przedstawienia kilku przebiegów w jednym oknie (np. prąd i napięcie w osobnych osiach).
- Automatyczny zapis wygenerowanego wykresu do pliku graficznego (`pdf` lub `png`) o wysokiej rozdzielczości.
- Dobór stylistyki wykresu (kolory, style linii, grubości) zapewniający czytelność przy druku czarno-białym.

Ocena 5.0 (Bardzo Dobry)

- Spełnienie wymagań na ocenę 4.5.
- **Analiza numeryczna sygnału:** Wykorzystanie NumPy do wyznaczenia parametrów takich jak: wartość średnia, odchylenie standardowe lub lokalizacja lokalnych maksimów sygnału.
- Wyświetlenie wyników analizy (np. wartości RMS) bezpośrednio na wykresie w formie adnotacji tekstowej (`plt.text` lub `plt.annotate`).
- Ostateczna weryfikacja koncepcji projektu końcowego – przedstawienie wstępnego wykresu, który będzie częścią dokumentacji technicznej projektu.

5. WSKAZÓWKI DLA STUDENTA

- **Wektoryzacja:** Unikaj pętli `for` przy operacjach na całych tablicach. Zamiast `for x in tab: res.append(sin(x))` stosuj `res = np.sin(tab)` . Jest to kluczowe dla wydajności.

- **Oś czasu:** Przy generowaniu sygnałów okresowych dobierz krok próbkowania zgodnie z twierdzeniem Nyquista, aby uniknąć aliasingu na wykresie.
- **Prezentacja danych:** Używaj metody `plt.tight_layout()`, aby zapobiec nakładaniu się etykiet osi w układach wielowykresowych.
- **Dokumentacja:** Korzystaj z oficjalnej galerii Matplotlib (matplotlib.org/stable/gallery), aby znaleźć inspiracje do zaawansowanych sposobów prezentacji danych inżynierskich.

KARTA ĆWICZENIA LABORATORYJNEGO NR 4

Przedmiot: Podstawy informatyki i programowania

Kierunek: Elektrotechnika

Temat: Projektowanie algorytmów inżynierskich i budowa modułów aplikacji. Warsztat projektowy i tworzenie dokumentacji technicznej.

Wymiar godzin: 4 godz. lekcyjne

Rodzaj zajęć: Laboratorium programistyczne (Projektowe)

1. CEL ĆWICZENIA

Celem zajęć jest praktyczna realizacja głównego modułu obliczeniowego aplikacji inżynierskiej oraz rozpoczęcie prac nad dokumentacją techniczną projektu. Ćwiczenie ma na celu symulację rzeczywistego procesu wytwarzania oprogramowania inżynierskiego, od implementacji algorytmu po opis teoretyczny zjawiska.

Cele szczegółowe obejmują:

- **Integrację zaawansowanych struktur programistycznych:** Student uczy się łączyć operacje wejścia/wyjścia (I/O) z zaawansowaną logiką obliczeniową i wizualizacją danych w ramach jednego projektu.
- **Implementację algorytmu dziedzinowego:** Przeniesienie teorii inżynierskiej (np. obliczenia obwodowe, symulacja procesów dynamicznych) na działający kod źródłowy.
- **Opracowanie dokumentacji technicznej:** Nabycie umiejętności opisu architektury programu, zastosowanych metod numerycznych oraz prezentacji wyników w formie sformalizowanego raportu (PDF/Word).
- **Zarządzanie cyklem życia projektu:** Planowanie etapów pracy, testowanie cząstkowych modułów oraz optymalizacja kodu pod kątem czytelności i stabilności.

2. WPROWADZENIE MERYTORYCZNE

Budowa profesjonalnej aplikacji inżynierskiej wymaga wyjścia poza prostą realizację zadań algorytmicznych. Kluczowe staje się podejście systemowe, w którym program jest narzędziem rozwiązującym konkretny problem techniczny, a dokumentacja stanowi nieodłączną część produktu.

Kluczowe aspekty techniczne:

- **Architektura modułowa:** Podział programu na logiczne części: moduł danych, moduł obliczeniowy oraz moduł raportujący. Pozwala to na łatwiejsze testowanie i modyfikację

poszczególnych elementów projektu.

- **Modelowanie matematyczne w kodzie:** Poprawna translacja wzorów fizycznych i matematycznych na instrukcje programistyczne, z uwzględnieniem warunków brzegowych i ograniczeń fizycznych (np. nieujemność rezystancji).
- **Standardy dokumentacji inżynierskiej:** Opis programu musi zawierać wstęp teoretyczny, schemat blokowy algorytmu (np. w standardzie ISO/normy technicznej) oraz instrukcję obsługi z przykładami weryfikacyjnymi.
- **User Experience (UX) w konsoli:** Projektowanie przejrzystego interfejsu użytkownika (menu, czytelne komunikaty błędów), który prowadzi operatora przez proces obliczeniowy.

3. TREŚĆ ZADANIA

Zadanie polega na intensywnej pracy nad projektem końcowym (indywidualnie lub w parach). Student musi wykazać się postępem prac w zakresie implementacji kluczowych funkcjonalności oraz przedstawić szkielet dokumentacji technicznej.

Etapy realizacji projektu na zajęciach:

1. **Implementacja rdzenia obliczeniowego (Core Logic):** Budowa funkcji realizujących główny cel inżynierski projektu (np. algorytm rozwiązujący układ równań, symulator ładowania kondensatora).
2. **Budowa interfejsu danych:** Opracowanie mechanizmu pobierania parametrów od użytkownika lub z pliku oraz walidacja tych danych przed przystąpieniem do obliczeń.
3. **Redakcja dokumentacji (Drafting):** Przygotowanie opisu teorii działania algorytmu oraz stworzenie schematu blokowego obrazującego logikę programu.

4. WYMAGANIA I KRYTERIA OCENY

Ocena 3.0 (Dostateczny)

- Zdefiniowanie ostatecznego tematu projektu i zakresu jego funkcjonalności.
- Implementacja szkieletu programu (pusta struktura funkcji, menu wyboru).
- Przygotowanie pliku dokumentacji (strona tytułowa i spis treści).
- Uzasadnienie wyboru technologii (Python lub C++) pod kątem rozwiązywanego problemu.

Ocena 3.5 (Dostateczny Plus)

- Spełnienie wymagań na ocenę 3.0.
- Implementacja podstawowych obliczeń inżynierskich w formie działającego prototypu.

- Opracowanie sekcji „Podstawy teoretyczne” w dokumentacji (opis fizyki/matematyki zjawiska).
- Poprawne wyświetlanie wyników cząstkowych w konsoli.

Ocena 4.0 (Dobry)

- Spełnienie wymagań na ocenę 3.5.
- **Integracja danych:** Program poprawnie wczytuje dane wejściowe i przekazuje je do modułu obliczeniowego.
- Stworzenie poprawnego schematu blokowego algorytmu głównego (np. w narzędziu Lucidchart, Draw.io).
- Wprowadzenie podstawowej obsługi błędów (np. blokada dzielenia przez zero).

Ocena 4.5 (Dobry Plus)

- Spełnienie wymagań na ocenę 4.0.
- **Kompletność modułów:** Program realizuje pełny cykl (Akwizycja -> Obliczenia -> Wizualizacja/Raport).
- Dokumentacja zawiera wstępny opis implementacji (opis funkcji, parametry wejścia/wyjścia).
- Kod źródłowy jest czytelny, modularny i udokumentowany komentarzami (tzw. Docstrings w Pythonie).

Ocena 5.0 (Bardzo Dobry)

- Spełnienie wymagań na ocenę 4.5.
- **Zaawansowanie inżynierskie:** Program rozwiązuje nietrywialny problem (np. analiza statystyczna dużego zbioru, symulacja z wieloma zmiennymi).
- Dokumentacja zawiera kompletny opis weryfikacji programu (przykładowe zestawy testowe i porównanie z wynikami oczekiwanymi).
- Implementacja zapisu wyników końcowych do pliku raportu (PDF lub tekstowy) oraz generowanie wykresu technicznego.

5. WSKAZÓWKI DLA STUDENTA

- **Modularność:** Nie pisz całego programu w jednym bloku kodu. Rozbij go na małe funkcje (np. `oblicz_parametry()`, `generuj_wykres()`). Ułatwi to debugowanie.
- **Schematy blokowe:** Pamiętaj o standardowych kształtach: prostokąt (operacja), romb (decyzja/warunek), równoległobok (wejście/wyjście danych).

- **Dokumentacja:** Używaj edytora równań (np. w MS Word) do zapisu wzorów fizycznych. Zrzuty ekranu z działania programu powinny być czytelne i opisane.
- **Zasada MVP:** Skup się najpierw na tym, aby algorytm liczył poprawnie (Minimum Viable Product), a dopiero później dopracowuj estetykę wykresów i interfejsu.

KARTA ĆWICZENIA LABORATORYJNEGO NR 5

Przedmiot: Podstawy informatyki i programowania

Kierunek: Elektrotechnika

Temat: Weryfikacja efektów uczenia się i obrona projektów inżynierskich. Finalna ocena rozwiązań programistycznych i dokumentacji technicznej.

Wymiar godzin: 2 godz. lekcyjne

Rodzaj zajęć: Laboratorium programistyczne (Zaliczeniowe)

1. CEL ĆWICZENIA

Celem zajęć jest końcowa weryfikacja zdobytych umiejętności w zakresie projektowania, implementacji i dokumentowania aplikacji inżynierskich. Laboratorium ma formę publicznej (lub indywidualnej) obrony projektu, podczas której student prezentuje działanie programu oraz wykazuje się znajomością zastosowanych struktur danych i algorytmów.

Cele szczegółowe obejmują:

- **Weryfikację samodzielności pracy:** Sprawdzenie autentyczności kodu poprzez sesję pytań technicznych dotyczących konkretnych fragmentów implementacji.
- **Ocenę jakości dokumentacji technicznej:** Analizę merytoryczną dostarczonego raportu (PDF/Word) pod kątem poprawności opisu teorii, algorytmu oraz wyników testów.
- **Prezentację wyników inżynierskich:** Nabycie umiejętności zwięzłego przedstawienia celu projektu, sposobu rozwiązywania problemu oraz interpretacji uzyskanych wykresów/danych.
- **Podsumowanie warsztatu programistycznego:** Końcowa ocena stylu kodowania, modularności oraz odporności aplikacji na błędy (walidacja danych wejściowych).

2. WPROWADZENIE MERYTORYCZNE

W profesjonalnym procesie inżynieryjnym samo stworzenie działającego prototypu oprogramowania jest niewystarczające. Równie istotna jest **walidacja i weryfikacja (V&V)** oraz stworzenie dokumentacji, która umożliwi innym inżynierom zrozumienie, serwisowanie i dalszy rozwój systemu.

Kluczowe aspekty oceny inżynierskiej:

- **Poprawność numeryczna:** Zgodność wyników generowanych przez program z przewidywaniami teoretycznymi lub danymi referencyjnymi.
- **Stabilność aplikacji:** Zachowanie programu w sytuacjach brzegowych (np. wprowadzenie wartości zero, brak pliku wejściowego, błędny format danych).

- **Przejrzystość architektury:** Ocena, czy kod jest podzielony na logiczne moduły (funkcje), co świadczy o dojrzałości programistycznej autora.
- **Jakość dokumentacji:** Ocena, czy opis techniczny pozwala na odtworzenie procesu obliczeniowego bez kontaktu z autorem (tzw. powtarzalność badań).

3. TREŚĆ ZADANIA

Zadanie polega na przeprowadzeniu obrony przygotowanego projektu końcowego. Student prezentuje gotowe rozwiązanie na stanowisku laboratoryjnym, demonstrując pełną funkcjonalność aplikacji oraz przekazując finalną wersję dokumentacji technicznej prowadzącemu.

Przebieg obrony projektu:

1. **Demonstracja funkcjonalna:** Uruchomienie programu i wykonanie pełnego cyklu obliczeniowego na przykładowych danych (Akwizycja -> Obliczenia -> Wizualizacja).
 2. **Code Review:** Krótka rozmowa z prowadzącym na temat struktury kodu. Student musi potrafić wskazać i wyjaśnić działanie kluczowych funkcji oraz pętli sterujących.
 3. **Weryfikacja dokumentacji:** Przegląd raportu technicznego ze szczególnym uwzględnieniem poprawności schematu blokowego oraz analizy błędów.
-

4. WYMAGANIA I KRYTERIA OCENY KOŃCOWEJ

Ocena 3.0 (Dostateczny)

- Program uruchamia się i wykonuje podstawowe obliczenia inżynierskie bez błędów krytycznych.
- Dostarczono uproszczoną dokumentację (cel projektu, opis działania, kod źródłowy).
- Student rozumie ogólną zasadę działania programu, ale ma trudności z wyjaśnieniem szczegółów implementacyjnych.

Ocena 3.5 (Dostateczny Plus)

- Spełnienie wymagań na ocenę 3.0.
- Dokumentacja zawiera poprawny opis teoretyczny zjawiska oraz podstawowy schemat blokowy.
- Program poprawnie formatuje wyniki (np. precyzja miejsc po przecinku).
- Zastosowano podstawowy podział na funkcje.

Ocena 4.0 (Dobry)

- Spełnienie wymagań na ocenę 3.5.
- **Wizualizacja:** Program generuje czytelne wykresy techniczne (np. Matplotlib), które są dołączone do dokumentacji.
- Dokumentacja jest przejrzysta, zawiera zrzuty ekranu z testów oraz opisane osie na wykresach.
- Student sprawnie porusza się po kodzie i potrafi wyjaśnić logikę zastosowanych instrukcji warunkowych i pętli.

Ocena 4.5 (Dobry Plus)

- Spełnienie wymagań na ocenę 4.0.
- **Kompletność i styl:** Program cechuje się wysoką modularnością (jasno zdefiniowane funkcje z parametrami).
- Dokumentacja zawiera szczegółowy opis weryfikacji algorytmu (porównanie z obliczeniami analitycznymi).
- Zaimplementowano obsługę sytuacji błędnych (np. komunikat o braku danych w pliku).

Ocena 5.0 (Bardzo Dobry)

- Spełnienie wymagań na ocenę 4.5.
- **Profesjonalizm:** Projekt rozwiązuje złożony problem inżynierski. Kod jest wzorcowo udokumentowany (komentarze, nazewnictwo).
- Dokumentacja ma charakter profesjonalnego raportu technicznego (schemat blokowy zgodny z normami, poprawny edytor równań, wnioski końcowe).
- Student wykazuje się pełną biegłością w wybranym języku (Python/C++), swobodnie modyfikując fragment kodu na prośbę prowadzącego.

5. WSKAZÓWKI DLA STUDENTA

- **Przygotowanie pliku:** Upewnij się, że wszystkie pliki niezbędne do uruchomienia programu (np. dane wejściowe `.csv`) znajdują się w jednym folderze z kodem.
- **Dokumentacja PDF:** Dokumentację końcową należy dostarczyć w formacie PDF, aby uniknąć problemów z formatowaniem wzorów i schematów na innym urządzeniu.
- **Obrona kodu:** Bądź gotowy na pytanie typu: „Co się stanie, jeśli zmienię tę wartość w pliku wejściowym?” lub „Za co odpowiada ta konkretna linia kodu?”.
- **Estetyka:** Pamiętaj, że w dokumentacji inżynierskiej jakość wykresów (opisy osi, jednostki) jest równie ważna jak poprawność samego algorytmu.

TECHNICZNY PRZEWODNIK DO LABORATORIUM

LAB 1 Temat: Akwizycja i przetwarzanie danych w strukturach tablicowych

1. STRUKTURA DANYCH I ALOKACJA

W inżynierii unikamy tzw. „magicznych liczb”. Rozmiar tablicy powinien być zarządzany centralnie.

Wzorzec deklaracji:

```
#include <iostream>

const int MAX_SAMPLES = 100; // Globalna stała określająca limit bufora

int main() {
    double sensorData[MAX_SAMPLES]; // Tablica statyczna na liczby rzeczywiste
    int actualSize;                  // Liczba faktycznie wprowadzonych danych

    // ... dalsza logika
}
```

2. SCHEMATY INTERAKCJI (PĘTLE)

Podstawowy wzorzec wczytywania danych z zabezpieczeniem przed przepełnieniem bufora.

Logika wczytywania:

- Pobierz od użytkownika informację, ile próbek chce wprowadzić.
- Sprawdź, czy `n` nie przekracza `MAX_SAMPLES`.
- Użyj pętli `for` do wypełnienia tablicy.

Struktura pętli operacyjnej:

```
for (int i = 0; i < actualSize; i++) {
    // Odwołanie do elementu: sensorData[i]
    // Tutaj: wczytywanie (cin) lub generowanie losowe
}
```

3. ARYTMETYKA I PRECYZJA (ZŁOTE ZASADY)

Najczęstszym błędem jest utrata części ułamkowej przy obliczaniu średniej.

Mechanizm rzutowania (Casting): Jeśli dzielisz `sum` (suma wszystkich elementów) przez `count` (liczba elementów), a obie zmienne są liczbami całkowitymi, wynik będzie ucięty.

- **Rozwiązanie:** `double average = static_cast<double>(sum) / count;`

Formatowanie wyjścia (Biblioteka `<iomanip>`): Aby wymusić stałą liczbę miejsc po przecinku:

```
#include <iomanip>
// ...
std::cout << std::fixed << std::setprecision(2);
std::cout << "Wynik obliczeń: " << average << " [V]" << std::endl;
```

4. LOGIKA ALGORYTMÓW STATYSTYCZNYCH (PSEUDOKOD)

A. Wyszukiwanie Ekstremów (Min/Max)

Błędna metoda: Inicjalizacja `max = 0`. **Poprawna metoda inżynierska:**

1. Przyjmij, że pierwszy element tablicy (`data[0]`) jest Twoim obecnym `max`.
2. Przejdź pętlą od drugiego elementu (`i = 1`) do końca.
3. Jeśli `data[i]` jest większe od Twojego obecnego `max` -> nadpisz `max`.

B. Algorytm Sortowania (Idea Bubble Sort)

Sortowanie polega na wielokrotnym przejściu przez tablicę i zamianie miejscami sąsiadujących elementów, jeśli są w złej kolejności.

- Kluczowa operacja (Swap):

```
double temp = data[i];
data[i] = data[i+1];
data[i+1] = temp;
```

- Wymaga dwóch zagnieżdżonych pętli `for`.

C. Wyznaczanie Mediany

Mediana zależy od parzystości zbioru N (po uprzednim posortowaniu!):

- Jeśli N jest nieparzyste: Indeks środkowy to $N / 2$.
- Jeśli N jest parzyste: Średnia z elementów o indeksach $(N/2) - 1$ oraz $N/2$.

5. MODULARYZACJA (FUNKCJE)

Dla oceny 5.0 program musi być podzielony na moduły. Pamiętaj, że tablica w parametrze funkcji „traci” informację o swoim rozmiarze.

Prototypy funkcji:

```
// Funkcja nie zwraca nic, modyfikuje tablicę wewnątrz
void fillArray(double arr[], int size);

// Funkcja zwraca obliczoną wartość typu double
double calculateMean(double arr[], int size);

// Funkcja sortująca (zmienia kolejność w oryginalnej tablicy)
void sortData(double arr[], int size);
```

6. GENERATOR LICZB PSEUDOŁOSOWYCH

Użyteczny do testowania algorytmów na dużych zbiorach danych.

Niezbędne komponenty:

- `#include <cstdlib>` oraz `#include <ctime>`
- `srand(time(NULL));` // Inicjalizacja ziarna (raz w main)
- `rand() % 100` // Losowanie liczby całkowitej 0-99
- **Losowanie double z zakresu [min, max]:** `double val = min + (static_cast<double>(rand()) / RAND_MAX) * (max - min);`

7. CHECKLISTA PRZED ODDANIEM PROJEKTU

1. **Indeksowanie:** Czy na pewno pętle kończą się na `i < actualSize`, a nie `i <= actualSize`?
2. **Inicjalizacja:** Czy zmienna `suma` została wyzerowana przed pętlą?
3. **Typy danych:** Czy do przechowywania wyników pomiarowych użyto typu `double`, a nie `int`?
4. **Czytelność:** Czy nazwy zmiennych mówią co przechowują?
5. **Interfejs:** Czy program wyświetla prośbę o dane, czy tylko miga kursorem?

LAB 2 Temat: Automatyzacja obróbki danych i operacje na plikach (I/O)

1. BEZPIECZNA OBSŁUGA PLIKÓW (Menedżer kontekstu)

W Pythonie unikamy ręcznego otwierania i zamykania plików. Stosujemy konstrukcję `with`.

Wzorzec odczytu i zapisu:

```
# Odczyt danych
with open('pomiarzy.txt', 'r', encoding='utf-8') as f_in:
    lines = f_in.readlines()

# Zapis raportu
with open('raport.txt', 'a') as f_out:
    f_out.write("Podsumowanie analizy:\n")
```

2. POTOK PRZETWARZANIA TEKSTU (String Pipeline)

Dane w plikach tekstowych są zawsze ciągami znaków (`string`).

Kluczowe metody:

- `.strip()` – usuwa białe znaki (w tym znak nowej linii `\n`).
- `.split(',')` – tnie linię tekstu na listę względem separatora.
- `.replace(' ', '')` – usuwa zbędne spacje.

Przykład logiki „oczyszczania”:

```
raw_line = " 12.45 ; 13.00 \n"
clean_line = raw_line.strip()
parts = clean_line.split(';')
final_data = float(parts[0])
```

3. DYNAMICZNE STRUKTURY DANYCH (Listy)

W przeciwieństwie do tablic C++, listy w Pythonie nie mają ustalonego rozmiaru.

Najważniejsze operacje:

- `lista = []` – inicjalizacja.
 - `.append(wartosc)` – dodanie elementu.
 - `sum(lista)`, `len(lista)`, `min(lista)`, `max(lista)` – funkcje wbudowane.
-

4. ODPORNOŚĆ PROGRAMU (Obsługa Wyjątków)

Zabezpieczenie programu przed błędami w danych wejściowych.

Struktura Try-Except:

```
try:
    with open('dane.txt', 'r') as f:
        # Logika wczytywania
except FileNotFoundError:
    print("Błąd: Nie znaleziono pliku!")
except ValueError:
    print("Błąd: Dane nienumeryczne!")
```

5. FORMATOWANIE RAPORTÓW (f-strings)

Precyzyjne formatowanie wyników inżynierskich.

Wzorce:

```
wynik = 12.3456789
print(f"Parametr: {nazwa_czujnika} | Wartość: {wynik:.2f} [V]")
print(f"{'Czas [s]':<10} | {'Napięcie [V]':<10}")
```

6. MODULARYZACJA (Funkcje i zwracanie danych)

Funkcje w Pythonie mogą zwracać wiele wartości jednocześnie (jako krotki).

```
def analizuj_pomiary(dane):
    srednia = sum(dane) / len(dane)
    rozstep = max(dane) - min(dane)
    return srednia, rozstep

sr, roz = analizuj_pomiary(moja_lista)
```

7. AUTOMATYZACJA I CZAS (Biblioteka datetime)

Dodawanie znacznika czasu do raportu technicznego.

```
from datetime import datetime
teraz = datetime.now()
data_str = teraz.strftime("%Y-%m-%d %H:%M:%S")

with open('raport.txt', 'a') as f:
    f.write(f"Analiza wykonana: {data_str}\n")
```

8. WSKAZÓWKI LOGICZNE DO LABORATORIUM NR 2

1. Iteracja po pliku: `for line in f:` jest bezpieczniejsze dla dużych plików.
2. Puste linie: Sprawdzaj: `if not line.strip(): continue`.
3. Kolejność: Wczytaj -> Oczyszczyć (`strip`) -> Podziel (`split`) -> Konwertuj (`float`) -> Dodaj.
4. Separatory: Używaj `replace(',', '.')` przed konwersją na float, jeśli dane używają przecinków.

LAB 3 Temat: Obliczenia numeryczne i wizualizacja danych (NumPy & Matplotlib)

1. SILNIK OBLICZENIOWY (NumPy) – ZASADA WEKTORYZACJI

W NumPy unikamy pętli `for` na rzecz operacji tablicowych.

Tworzenie dziedziny:

- `np.linspace(start, stop, num)` – zadana liczba punktów.
- `np.arange(start, stop, step)` – zadany krok przyrostu.

```
import numpy as np
t = np.linspace(0, 1, 1000)
u = 230 * np.sqrt(2) * np.sin(2 * np.pi * 50 * t)
```

2. STATYSTYKA I ANALIZA SYGNAŁÓW

Błyskawiczna analiza wektorów danych.

Kluczowe funkcje:

- `np.mean(data)`, `np.std(data)`, `np.max(data)`.

- `np.argmax(data)` – lokalizacja indeksu wartości maksymalnej.
 - **RMS:** `u_rms = np.sqrt(np.mean(u**2))`.
-

3. FUNDAMENT WIZUALIZACJI (Matplotlib)

Nakładanie warstw graficznych na płótno.

```
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 6))
plt.plot(t, u, label='Napięcie u(t)', color='blue')
```

4. STANDARDY WYKRESU INŻYNIERSKIEGO (Checklista)

- `plt.xlabel('Opis osi [Jednostka]')`.
 - `plt.ylabel('Opis osi [Jednostka]')`.
 - `plt.title('Tytuł wykresu')`.
 - `plt.grid(True)` oraz `plt.legend()`.
-

5. PORÓWNYWANIE DANYCH (Układy Subplots)

Prezentacja kilku przebiegów w jednym oknie na osobnych osiach.

```
fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True)
ax1.plot(t, u_in)
ax2.plot(t, u_out)
plt.tight_layout()
```

6. PREZENTACJA DANYCH DYSKRETNYCH (Pomiary)

- `plt.scatter(x, y)` – wykres punktowy.
 - `plt.plot(x, y, 'rx')` – czerwone krzyżyki.
-

7. EKSPORT I ADNOTACJE

- `plt.savefig('plik.pdf', dpi=300)`.
 - `plt.text(x, y, 'Opis punktu')` oraz `plt.axhline()` dla linii poziomych.
-

8. WSKAZÓWKI LOGICZNE (Inżynierskie Pro-Tips)

1. **Konwersja:** `data_np = np.array(lista_pythonowa)` .
2. **Aliasing:** Jeśli wykres jest "połamany", zwiększ gęstość punktów w linspace.
3. **Slicing:** `plt.plot(t[:100], u[:100])` pokazuje fragment sygnału.
4. **Filtrowanie:** `data[data > 0]` bez użycia pętli.

LAB 4 Temat: Architektura aplikacji, schematy blokowe i walidacja danych

1. ARCHITEKTURA MODUŁOWA (Separacja Logiki)

Podział programu na logiczne bloki funkcjonalne.

```
def wczytaj_konfiguracje(sciezka): pass
def algorytm_obliczeniowy(dane): pass
def generuj_raport(wyniki): pass

if __name__ == "__main__":
    dane = wczytaj_konfiguracje('input.csv')
    wyniki = algorytm_obliczeniowy(dane)
    generuj_raport(wyniki)
```

2. WALIDACJA DANYCH (Garda Inżynierska)

Weryfikacja fizycznej poprawności parametrów wejściowych.

```
def sprawdz_parametry(R, C):
    if R <= 0 or C <= 0:
        raise ValueError("Parametry muszą być dodatnie!")
```

3. TRANSLACJA MATEMATYKI NA KOD (Metody Numeryczne)

Wykorzystanie dyskretnych przybliżeń operacji matematycznych.

Przykład: `Q = np.trapz(prad, czas)` – całkowanie numeryczne metodą trapezów.

4. DOKUMENTACJA TECHNICZNA (Standardy Raportowania)

- Nazwa modułu i opis teoretyczny (wzory).
 - Parametry wejściowe oraz typ zwracanego wyniku.
-

5. SCHEMATY BLOKOWE (Logika Algorytmu)

- **Owal:** Start/Koniec.
 - **Równoległobok:** Dane We/Wy.
 - **Prostokąt:** Obliczenia.
 - **Romb:** Decyzja (warunek).
-

6. TESTOWANIE (Weryfikacja Wyników)

Porównanie wyniku programu z obliczeniami ręcznymi dla wybranego przypadku testowego (tzw. "Złoty Standard").

7. WSKAZÓWKI DO DOKUMENTACJI (Word / PDF)

- Edytor równań (`Alt + =`) dla wzorów matematycznych.
 - Numerowane podpisy pod rysunkami i wykresami.
 - Czytelne zrzuty ekranu kodu (czcionka stałej szerokości).
-

8. LOGICZNE PUŁAPKI PROJEKTOWE

1. Unikaj zmiennych globalnych.
 2. Jednostki w komunikatach `print` są obowiązkowe.
 3. Stosuj ścieżki względne, aby projekt był przenaszalny.
-

LAB 5 Temat: Walidacja końcowa, Code Review i standardy prezentacji technicznej

1. INTEGRALNOŚĆ PROJEKTU (File Structure)

Projekt musi działać bezpośrednio po skopiowaniu folderu.

Checklista:

1. Kod źródłowy.
 2. Pliki danych (`.csv`).
 3. Wygenerowany raport.
 4. Dokumentacja PDF.
-

2. STANDARYZACJA DOKUMENTACJI (Checklista jakości)

- Czy opis teoretyczny zawiera poprawne wzory?
 - Czy schemat blokowy odpowiada faktycznemu kodowi?
 - Czy załączono dowód poprawności obliczeń (testy)?
-

3. PROCEDURA CODE REVIEW (Obrona Kodu)

Pytania dotyczące walidacji, wyboru struktur danych oraz reakcji programu na zmianę parametrów.

4. STABILNOŚĆ I ODPORNOŚĆ (Błędy Krytyczne)

Zastosowanie bloków `try-except` w głównym punkcie wejścia aplikacji, aby uniknąć nagłego przerwania pracy systemu.

5. ESTETYKA INŻYNIERSKA (Wykresy i Raporty)

- Opisy osi wraz z jednostkami.
 - Czytelność czcionek i brak nakładania się elementów (`tight_layout`).
 - Wysoka rozdzielczość plików graficznych.
-

6. WERYFIKACJA SAMODZIELNOŚCI (Plagiaty)

- Spójne nazewnictwo zmiennych.
 - Komentarze opisujące logikę.
 - Zdolność do modyfikacji kodu "na żywo".
-

7. SCHEMAT PREZENTACJI PROJEKTU (Show & Tell)

Cel projektu -> Sposób akwizycji danych -> Algorytm obliczeniowy -> Prezentacja i interpretacja wyników.

8. KOŃCOWE WSKAZÓWKI TECHNICZNE

- Usuń pliki tymczasowe przed oddaniem.
- Sprawdź poprawność jednostek fizycznych.
- Zweryfikuj, czy uzyskane wyniki mieszczą się w granicach zdrowego rozsądku fizycznego.

DODATKOWY Temat: Tworzenie interfejsów graficznych (GUI) z wykorzystaniem biblioteki FreeSimpleGUI

1. WPROWADZENIE DO FREESIMPLEGUI

FreeSimpleGUI to biblioteka umożliwiająca szybkie tworzenie okienkowych aplikacji graficznych bez konieczności zaawansowanej znajomości programowania obiektowego. Pozwala ona na zastąpienie tekstowego interfejsu konsoli (`input()` , `print()`) interaktywnym oknem z przyciskami, polami tekstowymi i wykresami.

Instalacja biblioteki: `pip install FreeSimpleGUI`

2. ARCHITEKTURA OKNA (UKŁAD I PĘTLA ZDARZEŃ)

Tworzenie aplikacji w FreeSimpleGUI opiera się na dwóch filarach: definicji układu (listy list) oraz pętli zdarzeń, która nasłuchuje interakcji użytkownika.

Podstawowy wzorzec aplikacji inżynierskiej:

```

import FreeSimpleGUI as sg

# 1. Definicja układu okna (Layout) - każdy wiersz to osobna lista
layout = [
    [sg.Text("Parametry układu pomiarowego", font=("Helvetica", 14))],
    [sg.Text("Napięcie wejściowe [V]:"), sg.Input(key="-U-", size=(10, 1))],
    [sg.Text("Rezystancja obciążenia [ $\Omega$ ):"), sg.Input(key="-R-", size=(10, 1))],
    [sg.Button("Oblicz natężenie"), sg.Button("Zamknij")],
    [sg.HorizontalSeparator()],
    [sg.Text("Wynik obliczeń:"), sg.Text("", key="-OUT-", text_color="yellow", font=
("Helvetica", 12, "bold"))]]
]

# 2. Inicjalizacja okna
window = sg.Window("Analizator Inżynierski v1.0", layout)

# 3. Pętla zdarzeń (Event Loop)
while True:
    event, values = window.read() # Oczekiwanie na akcję użytkownika

    # Wyjście z programu
    if event in (sg.WIN_CLOSED, "Zamknij"):
        break

    # Logika po kliknięciu przycisku
    if event == "Oblicz natężenie":
        try:
            # Pobieranie wartości ze słownika 'values' za pomocą kluczy
            u = float(values["-U-"])
            r = float(values["-R-"])

            if r <= 0:
                sg.popup_error("Rezystancja musi być większa od zera!")
                continue

            wynik = u / r
            # Aktualizacja pola w oknie
            window["-OUT-"].update(f"Wynik: {wynik:.4f} A")

        except ValueError:
            sg.popup_error("Błąd: Wprowadź poprawne liczby!")

window.close()

```

3. KLUCZOWE ELEMENTY INTERFEJSU (WIDGETY)

- `sg.Text`: Wyświetlanie statycznych opisów i etykiet.

- **sg.Input:** Pole do wprowadzania danych przez użytkownika. Zawsze posiada parametr `key` do identyfikacji danych w słowniku `values`.
 - **sg.Button:** Przycisk wyzwalający zdarzenie (`event`) o nazwie równej tekstowi na przycisku.
 - **sg.FileBrowse:** Specjalistyczny przycisk otwierający systemowe okno wyboru pliku (idealny do Lab 2).
 - **sg.Multiline:** Pole do wyświetlania wielolinijkowych raportów tekstowych.
-

4. INTEGRACJA Z LOGIKĄ PROJEKTU

W profesjonalnym podejściu inżynierskim interfejs GUI powinien być oddzielony od logiki obliczeniowej. Zaleca się wywoływanie funkcji opracowanych w ramach Lab 2 i Lab 3 wewnątrz pętli zdarzeń okna.

Zalety stosowania FreeSimpleGUI w projekcie końcowym:

1. **Walidacja w czasie rzeczywistym:** Możliwość natychmiastowego sprawdzenia poprawności danych przed ich przetworzeniem.
 2. **Obsługa plików:** Łatwe wskazywanie ścieżek do plików `.csv` bez konieczności ręcznego wpisywania nazw w konsoli.
 3. **Interaktywność:** Możliwość zmiany parametrów symulacji "na żywo" i natychmiastowej aktualizacji wyników.
-

5. WSKAZÓWKI TECHNICZNE

- **Słownik `values`:** Wszystkie dane wpisane w pola `sg.Input` są zwracane jako ciągi znaków (string). Konieczna jest konwersja na `float` lub `int` przed obliczeniami.
- **Klucze (Keys):** Stosuj unikalne i czytelne nazwy kluczy (np. `"-TEMP_IN-"`), co ułatwia zarządzanie dużymi formularzami.
- **Komunikaty Popup:** Używaj `sg.popup("Informacja")` lub `sg.popup_error("Błąd")` do szybkiego informowania użytkownika o statusie pracy programu.